Reviewer Pack — Minimal Monomial Degree Invariant of Matroid Polynomials vs …

Entry 39665084879a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 20:40:41 UTC

Minimal Monomial Degree Invariant of Matroid Polynomials vs Permanent Circuit Size

Entry ID: `39665084879a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 20:40:41 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Complexity Theory: Permanent Circuit Complexity

Statement:

> ['For any matroid M with n elements, the minimal degree of a monomial in the matroid polynomial of M is Ω(2^{n/2 - O(1)}) compared to the size of a smallest circuit in a permanent-encoding circuit for M.', 'The minimal monomial degree invariant ρ_M of M satisfies ρ_M = Θ(2^{n/2}) for all n-element matroids, and this invariant is preserved under minor operations on the matroid.', 'For any constant c > 0, if there exists an n-element matroid with a permanent-encoding circuit size less than 2^{n/2 - c}, then ρ_M for that matroid must be less than 2^{n/2 - c + O(1)}.']

Rationale (proposer's reasoning):

> ['Matroid theory has been applied in geometry and optimization, yet its connection to complexity theory is largely unexplored. The invariant ρ_M could reveal the intrinsic structure of permanent encodings that traditional circuit complexity metrics do not capture.', 'If such an invariant can be shown to be preserved under minor operations and grow exponentially with the size of the matroid, it would suggest a fundamental difference between permanent and determinant computation.']

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a11ca845471dcb1c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all n-element matroids, the ratio between the minimal monomial degree invariant ρ_M and the permanent circuit size is greater than or equal to 2^(n/2 - O(1)) with at least 80% of the seeds producing a ratio within this range. The conjecture is falsified if any seed produces a ratio less than 2^(n/2 - O(1)).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `matroid theory AND permanent circuit complexity` - `monomial degree invariant matroid polynomial AND complexity theory` - `θ(2^(n/2)) minor operations matroid permanent encoding`

Top relevant hits considered: - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1608.04025v1] Quasi-matroidal classes of ordered simplicial complexes - [http://arxiv.org/abs/1903.10609v3] Circuit Complexity of Knot States in Chern-Simons theory - [http://arxiv.org/abs/1301.2045v3] Integral-valued polynomials over the set of algebraic integers of bounded degree - [http://arxiv.org/abs/2507.20131v1] Permanent Data Encoding (PDE): A Visual Language for Semantic Compression and Knowledge Preservation in 3-Character Unit - [http://arxiv.org/abs/1502.06896v2] On excluded minors of connectivity 2 for the class of frame matroids - [http://arxiv.org/abs/2501.17742v2] The minors of matroids with an adjoint

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matroid_polynomial(matroid):
        n = len(matroid)
        if n == 0:
            return {()}
        else:
            base_poly = matroid_polynomial(matroid[:-1])
            new_poly = {}
            for subset in base_poly:
                new_subset = tuple(sorted(subset + (n-1,)))
                new_poly[new_subset] = base_poly[subset]
            for i in range(n):
                if any(i in subset for subset in base_poly):
                    continue
                new_poly[(i,)] = 0
            return new_poly
```

```
    def permanent_circuit_size(matroid):
        n = len(matroid)
        if n == 0:
            return 1
        else:
            size = 0
            for i in range(n):
                if any(i in subset for subset in matroid):
                    continue
                new_matroid = [tuple(sorted(subset + (i,))) for subset in matroid]
                size += permanent_circuit_size(new_matroid)
            return size

    def min_monomial_degree(poly):
        if not poly:
            return 0
        return max(len(subset) for subset in poly)

    n = random.randint(5, 40)
    matroid = [tuple(random.sample(range(n), k)) for k in range(1, n)]

    mp = matroid_polynomial(matroid)
    pccs = permanent_circuit_size(matroid)
    mmd = min_monomial_degree(mp)

    ratio = mmd / pccs
    conjecture_holds = ratio >= 2 ** (n / 2 - 1)
    counterexample = "" if conjecture_holds else f"Ratio {ratio} < 2^{n/2-1}"

    return {
        "metric_name": "Minimal Monomial Degree Invariant",
        "metric_value": mmd,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] / permanent_circuit_size(run_trial(0)["metric_name"]) for r
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=NA support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Ratio < 2^{n/2-1}' first_failing_seed={first_fai
```

```
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1a6dcaf.py", line 80, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1a6dcaf.py", line 58, in run_trial
    mp = matroid_polynomial(matroid)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1a6dcaf.py", line 26, in matroid_polyn
    base_poly = matroid_polynomial(matroid[:-1])
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1a6dcaf.py", line 26, in matroid_polyn
    base_poly = matroid_polynomial(matroid[:-1])
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1a6dcaf.py", line 26, in matroid_polyn
    base_poly = matroid_polynomial(matroid[:-1])
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 28 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1a6dcaf.py", line 30, in matroid_polyn
    new_poly[new_subset] = base_poly[subset]
                           ~~~~~~~~~^^^^^^^^
TypeError: 'set' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the error in the test code to ensure it can run to completion and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15543
2	preregistration	ollama_remote	glm4:latest	0	0	10148
3	novelty	ollama_remote	glm4:latest	0	0	8157
4	novelty	ollama_remote	glm4:latest	0	0	9609
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18573
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11001
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10966
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8680
9	judge	ollama_remote	glm4:latest	0	0	11648

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 104326 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/39665084879a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/39665084879a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/39665084879a.tar.gz` (if generated)